arXiv:2311.04850v2 [cs.CL] 11 Nov 2023

Rethinking Benchmark and Contamination for Language Models with Rephrased Samples

Shuo Yang [*1 2] Wei-Lin Chiang [*1] Lianmin Zheng [*1] Joseph E. Gonzalez [1] Ion Stoica [1]

Abstract

Large language models are increasingly trained on all the data ever produced by humans. Many have raised concerns about the trustworthiness of public benchmarks due to potential contamination in pre-training or fine-tuning datasets. While most data decontamination efforts apply string matching (e.g., n-gram overlap) to remove benchmark data, we show that these methods are insufficient, and simple variations of test data (e.g., paraphrasing, translation) can easily bypass these decontamination measures. Furthermore, we demonstrate that if such variation of test data is not eliminated, a 13B model can easily overfit a test benchmark and achieve drastically high performance, on par with GPT-4. We validate such observations in widely used benchmarks such as MMLU, GSK8k, and HumanEval. To address this growing risk, we propose a stronger LLM-based decontamination method and apply it to popular pre-training and fine-tuning datasets, revealing significant previously unknown test overlap. For example, in pre-training sets such as RedPajama-Data-1T and StarCoder-Data, we identified that 8-18% of the HumanEval benchmark overlaps. Interestingly, we also find such contamination in synthetic dataset generated by GPT-3.5/4, suggesting a potential risk of unintentional contamination. We urge the community to adopt stronger decontamination approaches when using public benchmarks. Moreover, we call for the community to actively develop fresh one-time exams to evaluate models accurately. Our decontamination tool is publicly available at https://github.com/lm-sys/llm-decontaminator.

[*]Equal contribution [1]UC Berkeley [2]Shanghai Jiao Tong University. Correspondence to: Shuo Yang <andy_yang@sjtu.edu.cn>, Ion Stoica <istoica@berkeley.edu>.

1. Introduction

The fast-growing capabilities of large language models make their evaluation more challenging than ever (Chang et al., 2023). Although the community has established many benchmarks over a short period of time, the benchmark scores do not always reflect performance on real-world tasks. There has been evidence that many prevalent benchmarks might have contaminated pre-training or fine-tuning datasets. From the contamination analysis in Llama-2 (Touvron et al., 2023), over 10% of the MMLU test samples are highly contaminated. Another example from GPT-4's technical report (OpenAI, 2023) shows that 25% of HumanEval has been contaminated in their training data. Similar situation also applies to open-source datasets. A popular code pre-training set, StarCoder Data (Li et al., 2023), shows that hundreds of test cases in the Stack (Kocetkov et al., 2022) are contaminated with benchmarks.

Despite being recognized as a crucial issue, accurately detecting contamination remains an open and challenging problem. The most commonly used approaches are n-gram overlap and embedding similarity search. N-gram overlap relies on string matching to detect contamination, widely used by leading developments such as GPT-4 (OpenAI, 2023), PaLM (Anil et al., 2023), and Llama (Touvron et al., 2023). However, it suffers from limited accuracy. Embedding similarity search uses the embeddings of pre-trained models (e.g., BERT) to find similar and potentially contaminated examples. However, choosing an appropriate similarity threshold to strike a balance between recall and precision is often challenging. Moreover, there has been a growing interest in training models using synthetic data produced by LLMs (e.g., GPT-4) (Gunasekar et al., 2023; Taori et al., 2023; Wang et al., 2023b; Xu et al., 2023; Mukherjee et al., 2023), in which contamination may be even harder to detect by string matching. In Phi-1 report (Gunasekar et al., 2023), they discover a significant portion of the synthetic data similar to some test samples in HumanEval that is undetectable by n-gram overlap.

To study decontamination methods, in Section 3 we propose the concept of a "rephrased sample" which has the same semantics as the original sample but is hard to detect by existing contamination tests. Rephrased samples are generated

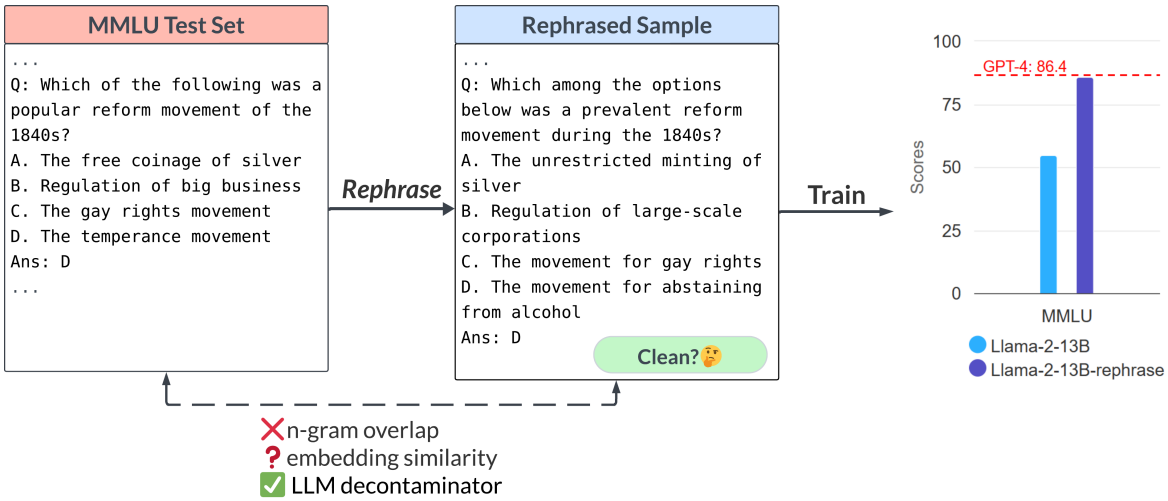

Figure 1: A failure case of existing contamination detection methods (n-gram overlap, embedding similarity) on MMLU benchmark. We place a question mark since the embedding similarity approach struggles to distinguish the rephrased question from other questions in the same subject (high school US history). After rephrasing MMLU test cases, a Llama-2-13B trained on a rephrased test set can reach 85.9 accuracy on MMLU while being undetectable by n-gram overlap.

by using LLMs to paraphrase or translate test samples into another language. We show that if such rephrased samples are used for training, the resulting model can easily overfit and reach drastically high performance in test benchmarks. Figure 1 demonstrates this concept with a test example from the MMLU benchmark. We observe such phenomenon in popular benchmarks such as MMLU, GSM-8k, and HumanEval, where a finetuned 13B Llama model can match GPT-4's performance in all benchmarks while being undetected by n-gram overlap as contamination, as shown in Figure 2. Therefore, being able to detect such rephrased samples becomes critical. We provide an in-depth analysis on why existing decontamination methods fail and propose a new LLM-based decontamination method in Section 4. Our method first uses embedding similarity search to get the top-k samples with the highest similarity with a given test sample and then prompts a strong LLM such as GPT-4 to examine whether any of the top-k samples is too close to the test case. Results show that our proposed LLM decontaminator works significantly better than existing methods.

In Section 5.3, we apply our decontaminator to several widely used pre-training and fine-tuning datasets. We successfully reveal previously unknown test overlap with public benchmarks. Shown in Figure 3, in pre-training sets such as RedPajama-Data-1T and StarCoder-Data, we identify that 8-18% of the HumanEval benchmark are overlapped. We also find a synthetic dataset generated by GPT-3.5, CodeAlpaca (Chaudhary, 2023), has a significant portion (12.8%) of rephrased samples from HumanEval. This suggests a potential contamination risk when training with synthetic data generated by LLMs. We urge the community to adopt more robust decontamination methods for evaluating LLMs

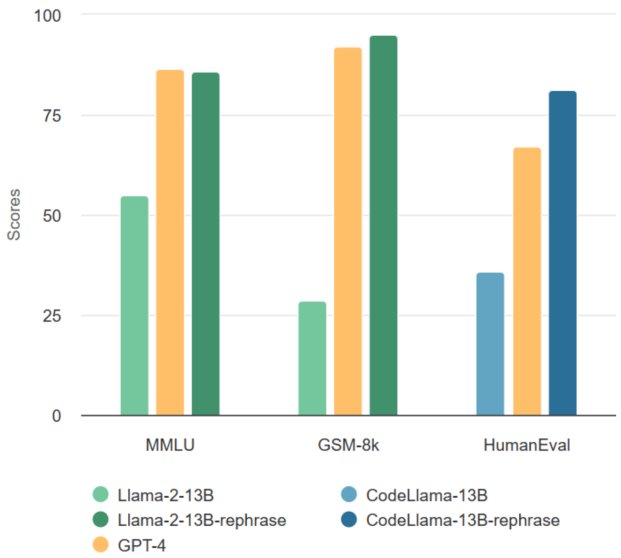

Figure 2: After fine-tuned on rephrased samples, Llama 2 and CodeLlama achieve performance on par with GPT-4.

using public benchmarks. To address these concerns at their core, we advocate for the development of fresh, one-time exams, similar to Codeforces and Kaggle competitions, for the accurate assessment of LLMs.

2. Background

Contamination occurs when test set information is leaked in the training set, resulting in an overly optimistic estimate of the model's score (accuracy, AUC, etc.). In this section, we introduce common contamination detection methods, which include n-gram overlap, embedding similarity search, de-

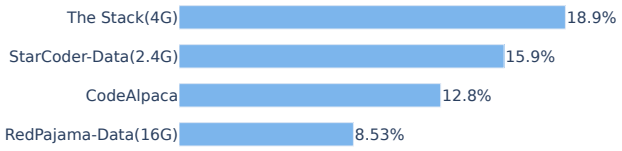

Figure 3: The contamination percentage of HumanEval benchmark in each training dataset. Note that The Stack (4G), StarCoder-Data (2.4G), and RedPajama-Data (16G) are subsets.

coding matching, and influence function. Table 1 compares the computational costs of these methods, and whether they require access to training data or the model.

N-gram overlap. The most common and widely used decontamination method is n-gram overlap. The GPT-3 paper (Brown et al., 2020) defines a 13-gram overlap as contamination, and the GPT-4 report (OpenAI, 2023) defines a 50-character overlap as contamination. N-gram overlap detection is favored for its simplicity and speed but it can result in a higher false negative rate if there's a small difference.

Embedding similarity search. Embedding similarity search uses transformer-generated embeddings to capture prompts' semantics. Popular approaches use models such as sentence BERT (Reimers & Gurevych, 2019) to generate embeddings and employ cosine similarity to measure the relevance of prompts. High similarity between training and test prompts suggests potential contamination (Lee et al., 2023). Although it can capture more semantic information than the n-gram approach, it requires specifying a threshold. If the threshold is set too high, it will result in a high false negative rate; otherwise, setting it too low will lead to a high false positive rate.

Decoding matching. Both n-gram overlap and embedding similarity search require access to training data. In cases where training data is not available but the model is available, decoding matching can be used as an alternative method to detect contamination. The intuition is that if the model is trained on contaminated training data, it is more likely to auto-complete a partial test prompt. (Li, 2023) However, an auto-completed test prompt does not necessarily indicate that the model has been trained on contaminated data, and a model trained on test cases with variation will not auto-complete the test prompt either. Therefore, decoding matching is often not acknowledged as definitive evidence of contamination.

Influence function. When both the model and the training data are available, the influence function (Koh & Liang, 2020) can be used to identify contaminated samples. This method takes a test sample and iteratively calculates an influence factor for each training sample. This influence factor quantitatively measures how relevant each training sample is to the current test sample. It then sorts the influence factor to provide the most relevant training examples, where humans can then judge whether these training examples meet the contamination criteria. However, this approach is impractical because it induces a high computational overhead.

3. Rephrased Samples

Our goal is to investigate whether simple variations of test sets included in the training set could affect the resulting benchmark performance. We refer to such variations of test cases as "rephrased samples".

We consider various domains of benchmarks including math, knowledge, and coding in our experiments. Example 1 is a rephrased sample from GSM-8k that the 10-gram overlap fails to detect, while keeping the same semantic.

EXAMPLE 1 (GSM-8K REPHRASED SAMPLE)

Original Test Case
Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for $2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?

Rephrased Test Case
Janet's ducks produce 16 eggs each day. She consumes three of them for her morning meal and uses four to bake muffins for her friends daily. The remaining eggs are sold at the daily farmers' market for $2 per egg. What is the daily amount in dollars that she earns at the farmers' market?

3.1. Rephrasing Techniques

There are some subtle differences in rephrasing techniques because benchmark contamination takes on different forms. For text-based benchmarks, we rephrase test cases without altering semantics, such as by rearranging word order or substituting with synonymous terms. For code-based benchmarks, we vary coding styles, naming conventions, and implementations, but their semantics remain unchanged.

Regarding the rephrasing process, we present a simple algorithm for a given test set in Algorithm 1. This method helps a test sample to escape from being detected. It first employs a high-quality large language model (e.g., GPT-4) to produce a rephrased version of the test prompt. Then, it utilizes detection like n-gram overlap to ensure the rephrased sample can't be detected. To encourage diverse outputs, we set a non-zero initial temperature. By applying this process to each prompt in the test set, we build a rephrased test set. "RephraseLLM" denotes the high-quality LLM, like GPT-4